

DöngüNet

Diagram Tutarsızlıklarının Çözümü,
Yazılım Planı ve Mermaid Diagram Kodları

Proje Adı	DöngüNet — Endüstriyel Simbiyoz Platformu
Takım Adı	VectorMatch
Doküman	Teknik Uyum ve Yazılım Planı
Ekip	3 kişi (Backend · Frontend · AI)
Ana Modüller	AI Eşleştirme · REST API · Web/Mobil UI
Hedef TRL	4 → 6-7 (Pilot OSB)
Teknoloji	Python 3.11 · FastAPI · Node.js/NestJS · PostgreSQL 16 + pgvector + PostGIS

Bu doküman; use case, ER, sequence ve class diyagramları arasındaki tutarsızlıkların tespitini, her biri için mimari kararı, veritabanı seviyesindeki migration SQL'lerini, kod tarafında yapılacak değişiklikleri ve düzeltilmiş 4 UML diyagramının Mermaid kaynak kodlarını içerir.

İçindekiler

1.	Yönetici Özeti	3
2.	Diyagram Tutarsızlıkları — Detaylı Çözüm	4
2.1	Cosine Benzerlik Eşiği (0.60 vs 0.65)	4
2.2	Proje ve Ekip Adı Standardizasyonu	5
2.3	UserRole ENUM Uyumsuzluğu	5
2.4	material_class Tipi (VARCHAR vs ENUM)	6
2.5	embeddings.record_id Tipi (UUID vs INTEGER)	7
2.6	Ekip Büyüklüğü Tutarsızlığı	8
2.7	AI Servisinin Stateless Tasarımı — Doğru Gösterimi	8
3.	Diyagramlardaki Eksikler ve Eklenecek Bileşenler	9
4.	Yazılım Tarafında Yapılacak İşler — Detaylı	11
4.1	AI Mikroservis (Hamid)	11
4.2	Backend (NestJS + PostgreSQL)	13
4.3	Frontend (Next.js + React Native)	15
4.4	DevOps ve Ortak Sorumluluklar	16
5.	8 Haftalık Yol Haritası	17
6.	Mermaid Diyagram Kodları	18
6.1	Use Case Diagram	18
6.2	ER Diagram (erDiagram)	20
6.3	Sequence Diagram	23
6.4	Class Diagram	26

1. Yönetici Özeti

DöngüNet, TEKNOFEST 2026 Sıfır Atık & Döngüsel Ekonomi Yarışması kapsamında geliştirilen, Organize Sanayi Bölgelerinde endüstriyel simbiyozu otomatikleştiren bir AI destekli platformdur. Sistem şu an TRL 4 (laboratuvar doğrulaması) seviyesinde; uçtan uca çalışan bir MVP mevcut. Ancak dört UML diyagramı (Use Case, ER, Sequence, Class) ile TEKNOFEST ön değerlendirme raporu arasında yedi kritik tutarsızlık ve on üç eksik bileşen tespit edilmiştir.

Bu doküman tutarsızlıkların her birinin (a) hangi belgede geçtiğini, (b) hangi kararın alınması gerektiğini, (c) veritabanı ve kod seviyesinde ne değişeceğini adım adım tanımlar. Ardından üç kişilik ekipteki (Backend, Frontend, AI/Hamid) her üye için ayrı ayrı görev listesi ve 8 haftalık sprint planı sunar. Son bölümde, düzeltilmiş dört UML diyagramının Mermaid kaynak kodları yer alır — bunlar mermaid.live veya benzeri araçlarda doğrudan render edilebilir.

Kritik Karar Özeti

Konu	Karar
Cosine eşiği	0.60 (ampirik teste dayanır)
Proje adı	DöngüNet (tek isim)
Ekip adı	VectorMatch (yarışma kaydı)
UserRole	ENUM('user', 'admin', 'osb_manager', 'api_key')
material_class	PostgreSQL ENUM (8 değer)
embeddings.record_id	UUID (backend ile birebir aynı)
AI servis	Stateless — DB'ye yazmaz
Eksik tablolar	notifications, weights_config, audit_log, carbon_factors, notifications_prefs
Human-in-the-Loop	Güven skoru < 0.80 → uzman onay kuyruğu

2. Diyagram Tutarsızlıkları — Detaylı Çözüm

Aşağıda yedi tutarsızlığın her biri için aynı şablon uygulanır: Tespit (nerede geçiyor), Karar (ne yapacağız), Migration/Kod Değişikliği (nasıl uygulanacak).

2.1 Cosine Benzerlik Eşiği (0.60 vs 0.65)

Belge	Yer	Değer
TEKNOFEST raporu	Bölüm 7, madde 3	0.60
Sequence diagram	Adım 2 (pgvector sorgusu)	0.65
Class diagram	Embedding sınıfı yan notu	0.65
ER diagram	matches tablosu yan notu	0.65

Karar

Eşik değeri 0.60 olarak sabitlenir. Gerekçe: 105 malzeme + 30 etiketli çift üzerinde yapılan ampirik testte 0.60 eşiği hem yanlış pozitif oranını düşük tuttu, hem de anlamlı eşleşmeleri kaçırmadı. Sequence, Class ve ER diyagramlarında bu değer güncellenir.

Kod Uygulaması

```
# ai-service/app/config.py
class Settings(BaseSettings):
    MATCH_THRESHOLD: float = 0.60 # cosine similarity, ampirik değer
    TOP_K_CANDIDATES: int = 20 # skarlama öncesi aday sayısı
    HITL_CONFIDENCE_THRESHOLD: float = 0.80 # Human-in-the-Loop tetiği

    class Config:
        env_file = ".env"

settings = Settings()

-- backend/migrations/002_add_config_table.sql
CREATE TABLE IF NOT EXISTS system_config (
    key          VARCHAR(100) PRIMARY KEY,
    value        JSONB NOT NULL,
    description   TEXT,
    updated_at    TIMESTAMP DEFAULT NOW(),
    updated_by    UUID REFERENCES users(id)
);

INSERT INTO system_config (key, value, description) VALUES
('match.threshold', '0.60::jsonb, 'Cosine benzerlik alt eşiği'),
('match.top_k', '20::jsonb, 'Skarlama öncesi aday sayısı'),
('match.hitl_threshold', '0.80::jsonb, 'Uzman onay tetik eşiği');
```

2.2 Proje ve Ekip Adı Standardizasyonu

Belge	Kullanılan Ad
UML diyagramlar	DöngüNet
TEKNOFEST rapor (proje)	EcoMatch
TEKNOFEST rapor (takım)	VectorMatch
Önceki iç yazışmalar	SymbioLoop

Karar

Proje adı: DöngüNet. Ekip adı: VectorMatch. Tüm dokümanlarda ve README'de bu iki isim tekilleştirilir. EcoMatch ve SymbioLoop deprecated. TEKNOFEST raporunun bir sonraki versiyonunda proje adı DöngüNet olarak güncellenir (yarışma başvurusu şu an "EcoMatch" ismiyle kayıtlı olabilir — jüriye sunumda "DöngüNet (namı diğer EcoMatch)" açıklaması eklenebilir).

2.3 UserRole ENUM Uyumsuzluğu

Tespit: ER diyagramında users.role ENUM olarak ('user','admin','osb','api_key') tanımlanmış; class diyagramında UserRole enum olarak USER, ADMIN, OSB_MANAGER, API_KEY tanımlanmış. 'osb' vs 'OSB_MANAGER' bir tutarsızlık; ayrıca 'api_key' bir rol mü yoksa authentication yöntemi mi olduğu belirsiz.

Karar

- Roller: user, admin, osb_manager
- api_key rol olarak kaldırılır — ayrı bir api_keys tablosuna taşınır.
- Class diagram enum değerleri lowercase snake_case yapılır (DB ile birebir eşleşme).

Migration

```
-- backend/migrations/003_fix_user_roles.sql

-- 1. Yeni ENUM tipini oluştur
CREATE TYPE user_role_new AS ENUM ('user', 'admin', 'osb_manager');

-- 2. Mevcut kayıtları taşı ('osb' -> 'osb_manager', 'api_key' -> ayrı tablo)
ALTER TABLE users ADD COLUMN role_new user_role_new;
UPDATE users SET role_new = CASE
  WHEN role::text = 'osb' THEN 'osb_manager'::user_role_new
  WHEN role::text = 'api_key' THEN 'user'::user_role_new -- default olarak user
  ELSE role::text::user_role_new
END;

-- 3. Eski kolonu değiştir
ALTER TABLE users DROP COLUMN role;
ALTER TABLE users RENAME COLUMN role_new TO role;
ALTER TABLE users ALTER COLUMN role SET NOT NULL;
ALTER TABLE users ALTER COLUMN role SET DEFAULT 'user';

DROP TYPE user_role;
ALTER TYPE user_role_new RENAME TO user_role;

-- 4. API anahtarları için ayrı tablo
CREATE TABLE api_keys (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  key_hash    VARCHAR(255) NOT NULL UNIQUE,
  name        VARCHAR(100) NOT NULL,
  scopes      TEXT[] DEFAULT ARRAY['read'],
```

```
    last_used    TIMESTAMP,  
    expires_at  TIMESTAMP,  
    created_at  TIMESTAMP DEFAULT NOW(),  
    revoked_at  TIMESTAMP  
);  
CREATE INDEX idx_api_keys_hash ON api_keys(key_hash) WHERE revoked_at IS NULL;
```

2.4 material_class Tipi (VARCHAR vs ENUM)

Tespit: ER diyagramında material_class VARCHAR(50), class diyagramında ise MaterialClass enum (METAL, PLASTIC, ORGANIC, CHEMICAL, TEXTILE, GLASS, PAPER, OTHER — toplam 8 değer). AI sınıflandırma prototiplerinin kategorileri sabit; VARCHAR olması yazım hatalarına ve tutarsız kayıtlara açık kapı bırakır (örn. 'metal' vs 'Metal' vs 'METAL').

Karar

PostgreSQL seviyesinde ENUM oluşturulur. Sınıflandırıcı çıktısı ile DB kaydı birebir eşleşir. Uygulama katmanında (NestJS + Python) da aynı enum sabitleri kullanılır.

Migration

```
-- backend/migrations/004_material_class_enum.sql
CREATE TYPE material_class AS ENUM (
  'metal',
  'plastic',
  'organic',
  'chemical',
  'textile',
  'glass',
  'paper',
  'other'
);

-- inputs tablosu
ALTER TABLE inputs ADD COLUMN material_class_new material_class;
UPDATE inputs SET material_class_new = LOWER(material_class)::material_class;
ALTER TABLE inputs DROP COLUMN material_class;
ALTER TABLE inputs RENAME COLUMN material_class_new TO material_class;
ALTER TABLE inputs ALTER COLUMN material_class SET NOT NULL;

-- outputs tablosu (aynı işlem)
ALTER TABLE outputs ADD COLUMN material_class_new material_class;
UPDATE outputs SET material_class_new = LOWER(material_class)::material_class;
ALTER TABLE outputs DROP COLUMN material_class;
ALTER TABLE outputs RENAME COLUMN material_class_new TO material_class;
ALTER TABLE outputs ALTER COLUMN material_class SET NOT NULL;

-- Index (kategori bazlı filtre için)
CREATE INDEX idx_outputs_material_class ON outputs(material_class);
CREATE INDEX idx_inputs_material_class ON inputs(material_class);
```

Python tarafında (AI servis)

```
# ai-service/app/schemas.py
from enum import Enum

class MaterialClass(str, Enum):
    METAL = "metal"
    PLASTIC = "plastic"
    ORGANIC = "organic"
    CHEMICAL = "chemical"
    TEXTILE = "textile"
    GLASS = "glass"
    PAPER = "paper"
    OTHER = "other"

class ClassifyResponse(BaseModel):
    material_class: MaterialClass
    confidence: float = Field(ge=0.0, le=1.0)
    top3: list[tuple[MaterialClass, float]]
    requires_human_review: bool # confidence < 0.80
```

TypeScript tarafında (backend)

```
// backend/src/materials/material-class.enum.ts
```

```
export enum MaterialClass {  
  METAL    = 'metal',  
  PLASTIC  = 'plastic',  
  ORGANIC  = 'organic',  
  CHEMICAL = 'chemical',  
  TEXTILE  = 'textile',  
  GLASS    = 'glass',  
  PAPER    = 'paper',  
  OTHER    = 'other',  
}
```


2.5 embeddings.record_id Tipi (UUID vs INTEGER)

Tespit: PostgreSQL şemasında embeddings.record_id UUID, ancak Python AI servisinin in-memory vektör store'unda record_id integer olarak tutuluyordu. Backend AI servisine POST ederken UUID string gönderiyor; AI servis bunu kabul etmeyip hata fırlatıyor veya sessizce bozuk index üretiyor.

Karar

Python AI servis stateless olduğu için aslında record_id'yi kendisi saklamamalı — sadece embedding vektörünü döndürüp geri gönderir. Backend bu vektörü record_id UUID ile birlikte pgvector tablosuna INSERT eder. Böylece tip uyumsuzluğu tamamen ortadan kalkar.

AI Servis — Doğru Kontrat

```
# ai-service/app/schemas.py
from pydantic import BaseModel, Field
from uuid import UUID
from typing import Optional

class EmbedRequest(BaseModel):
    text: str = Field(min_length=1, max_length=5000)
    record_id: Optional[UUID] = None          # sadece log/trace için, işlem için değil
    record_type: Optional[str] = None        # 'input' | 'output', sadece log

class EmbedResponse(BaseModel):
    vector: list[float] = Field(min_length=768, max_length=768)
    model: str = "all-mpnet-base-v2"
    dim: int = 768
    normalized: bool = True                  # DB'ye yazmadan önce backend bilsin

# ai-service/app/routers/embed.py
@router.post("/embed", response_model=EmbedResponse)
async def embed(req: EmbedRequest):
    text = enrich_text(req.text)
    vec = model.encode(text, normalize_embeddings=True).tolist()
    logger.info("embed", record_id=str(req.record_id), record_type=req.record_type)
    return EmbedResponse(vector=vec)
```

Backend — INSERT akışı

```
// backend/src/embeddings/embeddings.service.ts
async createEmbedding(recordId: string, recordType: 'input' | 'output', text: string) {
    const { vector } = await this.aiClient.post('/embed', {
        text,
        record_id: recordId,
        record_type: recordType,
    });

    // pgvector INSERT
    await this.db.query(
        `INSERT INTO embeddings (record_id, record_type, vector)
        VALUES ($1, $2, $3::vector)`,
        [recordId, recordType, `${vector.join(',')}`]
    );
}
```

2.6 Ekip Büyüklüğü Tutarsızlığı

Tespit: TEKNOFEST raporu Bölüm 13'te beş takım üyesi listelenmiş (Sehel Kayaoğlu, Esra Badur, Hamidreza Toutounchi, Umut Arda Tansever, Adil Efe). Ancak iç yazışmada teknik yazılım ekibi üç kişi (Backend + Frontend + AI/Hamid). Bu ayrım rapor içinde belirsiz.

Karar

Rapor Bölüm 13'te görev tanımları "Kod yazan" ve "Diğer" olarak ayrıştırılmalı. Öneri: yazılım geliştirme (kod yazan) 3 kişi; iş modeli, sunum, doküman ve pazar araştırması ekibi 2 kişi. Görev tablosu revize edilir — böylece jüri her üyenin somut katkısını görür.

Revize Görev Tablosu (öneri)

Üye	Ana Rol	Somut Çıktı
Hamidreza Toutounchi	AI Mikroservis Sahibi	FastAPI, SBERT, /embed, /classify, enrich_text, test dataset, AHP kalib
(Backend üye)	Backend Sahibi	NestJS REST API, PostgreSQL şema, pgvector, PostGIS, ScoringEngine
(Frontend üye)	Frontend Sahibi	Next.js web panel, React Native mobil, chatbot UI, OSB dashboard
Sehel Kayaoğlu	Proje Koordinatörü	Zaman planı, dokümantasyon, mimari kararlar, jüri sunumu
Esra Badur	İş Analizi & Süreç	Kullanım senaryoları, endüstriyel simbiyoz mantığı, maliyet analizi
Adil Efe	QA & Test	E2E testler, hata ayıklama, kullanıcı kabul testleri

Not: Yukarıdaki tablo bir öneridir. Ekibinizin gerçek rol dağılımına göre revize edilebilir. Önemli olan raporda "kim ne yazdı" sorusunun jüri gözünden net cevaplanmasıdır.

2.7 AI Servisinin Stateless Tasarımı — Doğru Gösterimi

Tespit: Class diyagramında `AIClient --produces--> Embedding` oku var. Bu, AI servisinin embedding'i "üretip sakladığı" yanlışlığına yol açıyor. Aslında AI servisi stateless: sadece vektörü döndürüyor, saklama sorumluluğu backend'de.

Karar

- AIClient sınıfının Embedding üretimi döndürme olarak işaretlenir: `getEmbedding(text): float[768]` — bir dönüş tipi, ilişki değil.
- Embedding varlığının oluşturucusu backend'deki `EmbeddingRepository / EmbeddingsService` olur — okla bu sınıfa çizilir.
- Yeni Class diyagramında (Bölüm 6.4) bu ilişki düzeltilmiş halde gösterilmiştir.

3. Diyagramlardaki Eksikler ve Eklenecek Bileşenler

Aşağıdaki bileşenler mevcut dört diyagramda ya hiç yok ya da eksik gösterilmiş. Bunların eklenmesi, rapordaki risk çözümlerinin (Human-in-the-Loop, Modüler Kural Motoru, RBAC vb.) mimari karşılığını sağlar.

3.1 ER Diagram — Eklenecek Tablolar

Tablo	Amaç	Kritiklik
notifications	Eşleşme kabul/red bildirimlerini kullanıcıya iletmek	Yüksek
weights_config	AHP kalibrasyonu — 5 faktör ağırlıklarını versiyonlamak	Yüksek
audit_log	DPP izlenebilirlik ve CBAM denetimi için kim/ne/ne zaman kayıtları	Yüksek
carbon_factors	CBAM için sektör bazlı emisyon faktörleri (mevzuat değişikliğiyle güncellenir)	Yüksek
human_review_queue	Güven skoru < 0.80 olan eşleşmelerin uzman onay kuyruğu	Yüksek
api_keys	API anahtarı yönetimi (bknz. bölüm 2.3)	Orta
facility_verification	Tesis doğrulama süreci (belge, onay tarihi)	Orta

Migration SQL (özet)

```
-- backend/migrations/005_missing_tables.sql

-- 1. notifications
CREATE TABLE notifications (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  type        VARCHAR(50) NOT NULL, -- 'match_accepted', 'match_rejected', 'review_required'
  title       VARCHAR(255) NOT NULL,
  body        TEXT,
  payload     JSONB,                -- ilgili match_id, output_id vb.
  read_at     TIMESTAMP,
  created_at  TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_notif_user_unread ON notifications(user_id) WHERE read_at IS NULL;

-- 2. weights_config (AHP kalibrasyonu)
CREATE TABLE weights_config (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  version     INTEGER NOT NULL,
  material    NUMERIC(4,3) NOT NULL CHECK (material BETWEEN 0 AND 1),
  quality     NUMERIC(4,3) NOT NULL CHECK (quality BETWEEN 0 AND 1),
  environmental NUMERIC(4,3) NOT NULL CHECK (environmental BETWEEN 0 AND 1),
  logistics   NUMERIC(4,3) NOT NULL CHECK (logistics BETWEEN 0 AND 1),
  economic    NUMERIC(4,3) NOT NULL CHECK (economic BETWEEN 0 AND 1),
  active      BOOLEAN DEFAULT FALSE,
  created_by  UUID REFERENCES users(id),
  created_at  TIMESTAMP DEFAULT NOW(),
  CHECK (ROUND(material+quality+environmental+logistics+economic, 3) = 1.000)
);
CREATE UNIQUE INDEX idx_weights_active ON weights_config(active) WHERE active = TRUE;
INSERT INTO weights_config (version, material, quality, environmental, logistics, economic, active)
VALUES (1, 0.30, 0.20, 0.20, 0.15, 0.15, TRUE);

-- 3. audit_log
CREATE TABLE audit_log (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  actor_id    UUID REFERENCES users(id),
  action      VARCHAR(50) NOT NULL, -- 'create', 'update', 'delete', 'accept', 'reject'
  entity      VARCHAR(50) NOT NULL, -- 'output', 'match', 'material_passport', ...
  entity_id   UUID NOT NULL,
  before      JSONB,
```

```
    after          JSONB,
    ip_address     INET,
    created_at     TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_audit_entity ON audit_log(entity, entity_id, created_at DESC);

-- 4. carbon_factors (CBAM)
CREATE TABLE carbon_factors (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    material_class  material_class NOT NULL,
    factor_type     VARCHAR(20) NOT NULL, -- 'virgin','secondary','transport'
    co2_per_kg      NUMERIC(10,4) NOT NULL, -- kg CO2e / kg malzeme
    source          VARCHAR(255),          -- 'Ecoinvent v3.10', 'EPA TRI' vb.
    valid_from      DATE NOT NULL,
    valid_to        DATE,
    created_at      TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_carbon_active ON carbon_factors(material_class, factor_type)
WHERE valid_to IS NULL;

-- 5. human_review_queue (HITL)
CREATE TABLE human_review_queue (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    match_id        UUID NOT NULL REFERENCES matches(id) ON DELETE CASCADE,
    confidence       NUMERIC(4,3) NOT NULL,
    reason          VARCHAR(255),
    status           VARCHAR(20) DEFAULT 'pending', -- pending, approved, rejected
    reviewed_by     UUID REFERENCES users(id),
    reviewed_at      TIMESTAMP,
    notes           TEXT,
    created_at      TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_review_pending ON human_review_queue(status, created_at)
WHERE status = 'pending';

-- matches tablosuna expires_at ekle
ALTER TABLE matches ADD COLUMN expires_at TIMESTAMP;
UPDATE matches SET expires_at = created_at + INTERVAL '30 days';
```

3.2 Use Case Diagram — Eklenecek Kullanım Durumları

Aktör	Yeni Use Case	Bağlantı
Tesis Kullanıcısı	Bildirimleri Görüntüle	match_accepted / rejected geldiğinde
Tesis Kullanıcısı	Profili Güncelle / Şifre Değiştir	Kimlik & Profil paketi
Tesis Kullanıcısı	Eşleştirme Geçmişini Görüntüle	matches tablosu üzerinden
Tesis Kullanıcısı	Eşleştirmeyi Reddet (gerekçe ile)	rejection_reason zorunlu
Sistem Admini	Kullanıcı Yönet (CRUD)	Kimlik & Profil paketi
Sistem Admini	API Anahtarı Yönet	api_keys tablosu
Sistem Admini	AHP Ağırlıklarını Kalibre Et	weights_config tablosu
Sistem Admini	Karbon Faktörlerini Güncelle	carbon_factors, CBAM için
Uzman (yeni aktör)	Onay Bekleyen Eşleşmeleri İncele	human_review_queue, HITL

3.3 Sequence Diagram — Eklenecek Alt Akışlar

- Kayıt & Doğrulama akışı: tesis kaydı → vergi no doğrulama → e-posta doğrulama → facilities.verified = true
- Eşleşme Reddetme akışı: Reddet butonu → rejection_reason zorunlu → karşı tarafa bildirim → audit_log kaydı
- Human-in-the-Loop akışı: güven skoru < 0.80 → human_review_queue'a INSERT → uzman incele → onay/red → notification
- Hata senaryoları: AI servis timeout/down → backend retry (3 kez) → fallback: eski cached embedding'lerle sadece keyword araması
- Bildirim akışı: match accepted → WebSocket/SSE ile frontend'e push + notifications tablosuna INSERT

3.4 Class Diagram — Eklenecek Sınıflar

Sınıf	Sorumluluk	Etkileşim
OSB	Organize Sanayi Bölgesi entitesi	Facility 1..* — kapsar
NotificationService	Bildirim üretme, kanal seçimi (in-app, Match, User)	Match, User
ChatbotService	Claude API çağırısı, context yönetimi	ChatMessage üretir
IoTHandler / MQTTSubscriber	Mosquitto'dan sensor verisi al, DB'ye yaz	SensorData üretir
ReportEngine	Environmental, CBAM, DPP PDF üretimi	Report üretir
ReviewQueueService	Human-in-the-Loop yönetimi	Match, User (Uzman)
RuleEngine	Modüler kural motoru (CBAM/DPP mevzuatı)	CBAM Calculator kullanır
AHPCalibrationService	Weights_config güncelleme, 50 eşleşme için	ScoringEngine ile eş

4. Yazılım Tarafında Yapılacak İşler — Detaylı

Aşağıdaki her bölüm bir ekip üyesinin sorumluluğuna denk gelir. Her görevin yanında öncelik (K=Kritik, Y=Yüksek, O=Orta, D=Düşük) ve tahmini süre (adam-gün) belirtilmiştir.

4.1 AI Mikroservis — Hamid

4.1.1 Tutarsızlık Düzeltmeleri

#	Görev	Öncelik	Süre
1	settings.py — MATCH_THRESHOLD=0.60, HITL_THRESHOLD=0.80 sabit hale getir	K	0.5
2	EmbedRequest / EmbedResponse Pydantic şemalarını UUID kabul edecek şekilde güncelle	K	0.5
3	MaterialClass enum tanımını Python tarafında yap (schemas.py)	K	0.5
4	In-memory vektör store'u tamamen kaldır — AI servis stateless olsun	K	1
5	enrich_text() sözlüğünü v2.0 olarak versiyonla (Git tag)	Y	0.5

4.1.2 Yeni Geliştirmeler

#	Görev	Öncelik	Süre
6	/classify endpoint'ini production'a bağla, ClassifyResponse şeması ile dön	K	2
7	Güven skoru hesabı: en yüksek 3 kategori softmax → requires_human_review flag'i	K	1
8	Embedding cache: Redis'e SHA256(text) key'i ile 24 saat cache	Y	2
9	Test veri seti genişletme: kategori-içi negatif çiftler (min 20), kısa gerçekçi girdiler (min 30), 2 yeni sektör (m	K	3
10	AHP kalibrasyon script'i (offline batch): backend'den son 50 eşleşmeyi çek, pairwise comparison → yeni ağırl	K	3
11	Model versiyonlama: /health endpoint'i model_name, model_version, din	O	0.5
12	Prometheus metrikleri: embed_duration_seconds, classify_confidence_bucket	O	1
13	Docker image slimming: multi-stage build ile <500 MB	D	1

4.1.3 Test Coverage Hedefleri

- Birim test (pytest): enrich_text, build_text, softmax güven hesabı → %90+ coverage
- Entegrasyon testi: /embed, /classify, /health endpoint'leri için happy + error path
- Doğruluk testi: 105+ malzeme, 30+ etiketli çift üzerinde precision/recall @ 0.60 eşiği
- Load test: 100 concurrent /embed isteği, p95 < 500ms

4.1.4 AI Servis Klasör Yapısı (öneri)

```

ai-service/
├── app/
│   ├── main.py           # FastAPI app
│   ├── config.py         # Settings, environment
│   ├── schemas.py        # Pydantic modelleri (EmbedRequest/Response, ClassifyResponse)
│   ├── models/
│   │   ├── __init__.py
│   │   └── sbert.py       # SBERT singleton yükleme
│   ├── services/
│   │   ├── embedder.py    # embed(text) -> vector
│   │   ├── classifier.py  # classify(text) -> ClassifyResponse
│   │   ├── enrichment.py  # enrich_text(text), Türkçe endüstri sözlüğü
│   │   └── cache.py       # Redis embedding cache
│   ├── routers/
│   │   ├── embed.py       # POST /embed, POST /embed/passport
│   │   ├── classify.py    # POST /classify
│   │   └── health.py      # GET /health
│   └── utils/
│       ├── logging.py     # structlog
│       └── metrics.py     # Prometheus
├── tests/
│   ├── conftest.py
│   ├── test_embedder.py
│   ├── test_classifier.py
│   ├── test_enrichment.py
│   ├── test_endpoints.py
│   └── data/
│       └── golden_dataset.jsonl # 105+ malzeme
├── scripts/
│   ├── ahp_calibration.py  # offline AHP hesaplama
│   └── evaluate_accuracy.py # precision/recall raporu
├── Dockerfile
├── docker-compose.yml
├── pyproject.toml         # ya da requirements.txt
└── README.md

```

4.2 Backend — NestJS + PostgreSQL

4.2.1 Migration ve Şema Düzeltmeleri

#	Görev	Öncelik	Süre
1	Migration 002-005: system_config, user_role fix, material_class ENUM, etkisiz tabloları kaldır	K	2
2	Seed data: default weights_config, carbon_factors ilk yükleme (Ecoinvent v3.10)	K	1
3	pgvector HNSW index parametrelerini tune et (m=16, ef_construction=64)	K	1
4	PostGIS index: facilities.location için GIST index	Y	0.5

4.2.2 Servis Katmanı

#	Görev	Öncelik	Süre
5	ScoringEngine: weights'i weights_config tablosundan oku (hardcode DEĞİL)	K	2
6	CBAMCalculator: emission factor'leri carbon_factors'tan oku, valid_to kontrolü	K	2
7	DPPGenerator: PDF üretimi (pdfkit/puppeteer), QR kod (qrcode)	K	2
8	NotificationService: in-app (DB) + WebSocket push (Socket.IO)	Y	2
9	ReviewQueueService: HITL — güven skoru < 0.80 için otomatik kuyruğa al	Y	2
10	AuditInterceptor: tüm mutation endpoint'lerini audit_log'a yaz	Y	1

#	Görev	Öncelik	Süre
11	ChatbotProxy: Claude API çağrısını backend'den yap (API key gizli)	Y	1.5
12	MQTTSubscriber: Mosquitto'ya bağlan, sensor_data'ya yaz	O	2
13	RuleEngine: CBAM/DPP kurallarını JSON config'ten yükle	O	3

4.2.3 REST Endpoint'leri (özet)

```
# Auth
POST /v1/auth/register      # yeni tesis kaydı
POST /v1/auth/login         # JWT döndürür
POST /v1/auth/refresh       # refresh token
POST /v1/auth/verify-email  # e-posta doğrulama

# Facilities
GET /v1/facilities/me
PATCH /v1/facilities/me
POST /v1/facilities/verify  # admin onayı

# Materials
POST /v1/materials/outputs  # çıktı/yan ürün kaydı
POST /v1/materials/inputs   # girdi ihtiyacı
GET /v1/materials/passport/:id # DPP JSON
GET /v1/materials/passport/:id/pdf
GET /v1/materials/passport/:id/qr

# Matches
GET /v1/matches/find/:outputId # eşleştirme talebi
POST /v1/matches/:id/accept
POST /v1/matches/:id/reject    # body: { reason }
GET /v1/matches/history

# Reports
GET /v1/reports/environmental/:matchId
GET /v1/reports/cbam/:matchId
GET /v1/reports/dpp/:passportId

# OSB Dashboard
GET /v1/osb/stats            # agregasyon
GET /v1/osb/regional-report
GET /v1/osb/facilities       # filtrelenebilir liste

# Admin
POST /v1/admin/users
GET /v1/admin/review-queue   # HITL kuyruğu
POST /v1/admin/review-queue/:id/approve
POST /v1/admin/weights       # yeni AHP ağırlıkları
POST /v1/admin/carbon-factors # CBAM güncelleme

# Notifications
GET /v1/notifications
PATCH /v1/notifications/:id/read
WebSocket /v1/notifications/stream

# AI Proxy (sadece internal)
POST /v1/internal/ai/embed    # AI servise proxy
POST /v1/internal/ai/classify

# Chatbot
POST /v1/chat                 # Claude API proxy'si
```

4.3 Frontend — Next.js + React Native

4.3.1 Web Panel (Next.js 14)

#	Görev	Öncelik	Süre
1	Auth ekranları: kayıt, giriş, şifre sıfırlama, e-posta doğrulama	K	2
2	Malzeme kayıt formu: çıktı ekle, canlı sınıflandırma preview (AI /classify)	K	3
3	Eşleştirme sonuç ekranı: skor kartları, breakdown grafiği (recharts), harita	K	3
4	DPP görüntüleme: QR kod, PDF indirme butonu	K	1

#	Görev	Öncelik	Süre
5	Bildirim merkezi: dropdown + gerçek zamanlı badge (WebSocket)	Y	2
6	Eşleştirme geçmişi tablosu: filtre, sıralama, export CSV	Y	2
7	OSB dashboard: Leaflet harita, agregasyon grafikleri, ısı haritası	Y	4
8	Chatbot widget: sağ alt köşe, Claude entegrasyonu backend proxy üzerinden	O	2
9	Admin paneli: kullanıcı yönetimi, review queue, weights kalibrasyon UI	O	3
10	i18n: Türkçe/İngilizce (next-intl)	D	1.5

4.3.2 Mobil (React Native)

#	Görev	Öncelik	Süre
11	Auth ekranları (aynı endpoint'ler)	Y	1.5
12	Saha malzeme kaydı: kamera ile foto çekme, GPS ile lokasyon	Y	3
13	QR kod tarayıcı: DPP doğrulama için	Y	2
14	Push notification: FCM/APNs entegrasyonu	Y	2
15	Eşleştirme listesi ve kabul/red	O	2
16	Offline mode: temel malzeme kaydı, sonra sync	D	3

4.4 DevOps ve Ortak Sorumluluklar

#	Görev	Sahip	Öncelik	Süre
1	OpenAPI contract dosyaları (api-contracts/ klasörü)	Backend + AI	K	2
2	Docker Compose: postgres+pgvector+postgis, redis, duckdb, redisquitto, ai, backend, frontend	Backend	K	1
3	GitHub Actions: CI (lint + test), CD (staging deploy)	Backend	Y	2
4	.env.example dosyası + secrets yönetimi (docker secrets/vault)	Backend	Y	0.5
5	Nginx reverse proxy config (SSL, rate limit)	Backend	Y	1
6	Log agregasyon: Loki + Grafana (opsiyonel: ELK)	Backend	O	2
7	Prometheus + Grafana dashboard (embed latency)	Backend	O	2
8	Backup stratejisi: pg_dump nightly, S3'e upload	Backend	O	1
9	E2E test: Playwright ile happy path	Frontend + Backend	Y	3
10	README + API docs (Swagger UI) + deployment guide	Backend	Y	2

4.5 API Contract Örneği (embed endpoint)

İki ekip üyesi arasındaki her sınırdaki tek gerçek kaynağı OpenAPI dosyası olmalı. Örnek: api-contracts/ai-service.openapi.yaml

```
openapi: 3.1.0
info:
  title: DöngüNet AI Service
  version: 1.0.0
paths:
  /embed:
    post:
      summary: Metin -> 768D vektör
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/EmbedRequest'
      responses:
        '200':
          description: Başarılı
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/EmbedResponse'
components:
  schemas:
    EmbedRequest:
      type: object
      required: [text]
      properties:
        text: { type: string, minLength: 1, maxLength: 5000 }
        record_id: { type: string, format: uuid, nullable: true }
        record_type:
          type: string
          enum: [input, output]
          nullable: true
    EmbedResponse:
      type: object
      required: [vector, model, dim, normalized]
      properties:
        vector:
```

```
type: array
items: { type: number }
minItems: 768
maxItems: 768
model:    { type: string, example: "all-mpnet-base-v2" }
dim:      { type: integer, example: 768 }
normalized: { type: boolean, example: true }
```

5. 8 Haftalık Yol Haritası

Hafta	Ana Odak	AI (Hamid)	Backend	Frontend
1	Diyagram + Şema Temizliği	Modeli (tut. düzeltme)	4.2.1 (migration 002-005)	API contract review
2	AI Kontrat + Sınıflandırma	/classify prod bağlama, güncelleme	AI endpoint, seed data	Malzeme kayıt formu (canlı classify)
3	Skorlama + CBAM	AHP kalibrasyon script islemleri	ScoringEngine + CBAMCalculation	Eşleştirme B2B Geçerliliği
4	Bildirim + HITL	Test dataset genişletme	NotificationService + Review	Bilgi Üretme Servisi + Aceli Soru Receptor
5	OSB Dashboard + Rapor	Prometheus metrikleri	Aggregation endpoint'leri	OSB Dashboard (harita + grafik)
6	Chatbot + Mobil	Model versiyonlama, embedding	Chatbot Engine (Claude API)	Chatbot widget + Mobil temel akış
7	Test + Optimizasyon	Doğruluk testi raporu (pre-release)	Loisiretecpoll vector tuning, E2E	E2E test, mobil QR tarayıcı
8	Pilot Hazırlık	"Sessiz Pilot" simülasyon	Deployment (staging→prod)	Kullanıcı dokümantasyonu, jüri sunu

Milestone'lar

- Hafta 2 sonu: Tüm diyagram tutarsızlıkları giderilmiş, migration'lar staging'de çalışır. /classify endpoint live.
- Hafta 4 sonu: Bir tesis kaydı → çıktı ekleme → eşleştirme → kabul → bildirim akışı end-to-end çalışır.
- Hafta 6 sonu: OSB dashboard, chatbot, mobil MVP hazır. Demo yapılabilir.
- Hafta 8 sonu: Pilot OSB'ye "Sessiz Pilot" için hazır sistem.

6. Mermaid Diyagram Kodları

Aşağıdaki dört diyagram, 2. bölümdeki tutarsızlıklar çözülmüş ve 3. bölümdeki eksikler eklenmiş haliyle Mermaid formatında verilmiştir. Bunları <https://mermaid.live> veya benzeri araçlarda (VSCode Mermaid Preview, Markdown editörler) doğrudan render edebilirsin. Çıktı SVG/PNG olarak dışa aktarılıp raporun ekine konulabilir.

Not: Mermaid'in sınırlamaları nedeniyle bazı görsel özellikler (renkli paket kutuları, «extend» ok stili gibi) birebir taşınamayabilir. Yapı ve içerik doğrudur.

6.1 Use Case Diagram

Mermaid use case diyagramı için standart bir syntax yok; en yaygın yaklaşım flowchart LR ile aktör → use case ilişkisini modellemektir. Alternatif olarak PlantUML kullanmak istersen aynı bilgiyi PlantUML syntax'inde de altta ekledim.

Seçenek A — Mermaid (flowchart tabanlı)

```
flowchart LR
    %% ==== Aktörler ====
    TK((Tesis<br/>Kullanıcısı))
    OSB((OSB<br/>Yöneticisi))
    ADM((Sistem<br/>Admini))
    UZM((Uzman<br/>HITL))
    IOT((IoT Sensör<br/>«system»))
    CLD((Claude API<br/>«external»))

    subgraph KP[Kimlik & Profil]
        UC1[Tesis Kaydı Yap]
        UC2[Tesis Doğrula]
        UC3[Giriş Yap - JWT]
        UC4[Profili Güncelle]
        UC5[Şifre Sıfırla]
    end

    subgraph MY[Malzeme Yönetimi]
        UC10[Çıktı/Yan Ürün Kaydet]
        UC11[Girdi İhtiyacı Tanımla]
        UC12[Malzeme Pasaportu Oluştur - DPP]
        UC13[QR Kod Üret]
    end

    subgraph AI[AI Eşleştirme]
        UC20[Eşleştirme Talep Et]
        UC21[Vektör Üret - SBERT]
        UC22[5 Faktörlü Skor Hesapla]
        UC23[Eşleştirmeyi Kabul Et]
        UC24[Eşleştirmeyi Reddet]
        UC25[Eşleştirme Geçmiş]
    end

    subgraph RP[Raporlama]
        UC30[Çevresel Etki Raporu Al]
        UC31[CBAM Raporu İndir - PDF]
        UC32[DPP Belgesi İndir - PDF]
    end

    subgraph YM[Yardımcı Modüller]
        UC40[Chatbot ile Konuş]
        UC41[Atık Öngörüsü Görüntüle]
        UC42[Sensör Verisi Gönder]
        UC43[Bildirimleri Görüntüle]
    end

    subgraph OY[OSB Yönetimi]
        UC50[OSB Dashboard]
        UC51[Toplu İstatistik Al]
    end
```

```
        UC52[Bölgesel Rapor Üret]
    end

    subgraph AD[Admin İşlemleri]
        UC60[Kullanıcı Yönet]
        UC61[API Anahtarı Yönet]
        UC62[AHP Ağırlıklarını Kalibre Et]
        UC63[Karbon Faktörlerini Güncelle]
    end

    subgraph HL[HITL - Human in the Loop]
        UC70[Onay Bekleyen Eşleşmeleri İncele]
    end

    %% ==== Aktör bağlantıları ====
    TK --> UC1
    TK --> UC3
    TK --> UC4
    TK --> UC5
    TK --> UC10
    TK --> UC11
    TK --> UC20
    TK --> UC23
    TK --> UC24
    TK --> UC25
    TK --> UC30
    TK --> UC31
    TK --> UC32
    TK --> UC40
    TK --> UC41
    TK --> UC43

    IOT --> UC42
    OSB --> UC50
    OSB --> UC51
    OSB --> UC52
    OSB --> UC3

    ADM --> UC2
    ADM --> UC3
    ADM --> UC60
    ADM --> UC61
    ADM --> UC62
    ADM --> UC63

    UZM --> UC70
    UZM --> UC3

    %% ==== include / extend ilişkileri ====
    UC10 -->|include| UC12
    UC12 -->|include| UC13
    UC20 -->|include| UC21
    UC20 -->|include| UC22
    UC22 -->|extend HITL| UC70
    UC40 -->|uses| CLD
    UC31 -->|extend| UC30

    %% ==== Stil ====
    classDef actor fill:#E8F5E9,stroke:#1B5E20,stroke-width:2px
    classDef ext fill:#FFF3E0,stroke:#E65100,stroke-width:2px
    class TK,OSB,ADM,UZM actor
    class IOT,CLD ext
```

Seçenek B — PlantUML (daha standart use case gösterimi)

Mermaid'in use case desteği zayıf; jüriye daha profesyonel görünen bir çıktı için PlantUML tercih edebilirsin. plantuml.com/plantuml adresinden render alabilirsin.

```
@startuml
left to right direction
skinparam actorStyle awesome
skinparam packageStyle rectangle

actor "Tesis Kullanıcısı" as TK
actor "OSB Yöneticisi" as OSB
actor "Sistem Admini" as ADM
actor "Uzman (HITL)" as UZM
actor "IoT Sensör" as IOT << system >>
actor "Claude API" as CLD << external >>

rectangle "DöngüNet Platformu" {

    package "Kimlik & Profil" {
        usecase "Tesis Kaydı Yap" as UC1
        usecase "Tesis Doğrula" as UC2
        usecase "Giriş Yap (JWT)" as UC3
        usecase "Profili Güncelle" as UC4
        usecase "Şifre Sıfırla" as UC5
    }

    package "Malzeme Yönetimi" {
        usecase "Çıktı/Yan Ürün Kaydet" as UC10
        usecase "Girdi İhtiyacı Tanımla" as UC11
        usecase "Malzeme Pasaportu Oluştur (DPP)" as UC12
        usecase "QR Kod Üret" as UC13
    }

    package "AI Eşleştirme" {
        usecase "Eşleştirme Talep Et" as UC20
        usecase "Vektör Üret (SBERT)" as UC21
        usecase "5 Faktörlü Skor Hesapla" as UC22
        usecase "Eşleştirmeyi Kabul Et" as UC23
        usecase "Eşleştirmeyi Reddet" as UC24
        usecase "Eşleştirme Geçmişi" as UC25
    }

    package "Raporlama" {
        usecase "Çevresel Etki Raporu Al" as UC30
        usecase "CBAM Raporu İndir (PDF)" as UC31
        usecase "DPP Belgesi İndir (PDF)" as UC32
    }

    package "Yardımcı Modüller" {
        usecase "Chatbot ile Konuş" as UC40
        usecase "Atık Öngörüsü Görüntüle" as UC41
        usecase "Sensör Verisi Gönder" as UC42
        usecase "Bildirimleri Görüntüle" as UC43
    }

    package "OSB Yönetimi" {
        usecase "OSB Dashboard Görüntüle" as UC50
        usecase "Toplu İstatistik Al" as UC51
        usecase "Bölgesel Rapor Üret" as UC52
    }

    package "Admin İşlemleri" {
        usecase "Kullanıcı Yönet" as UC60
        usecase "API Anahtarı Yönet" as UC61
        usecase "AHP Ağırlıklarını Kalibre Et" as UC62
        usecase "Karbon Faktörlerini Güncelle" as UC63
    }

    package "HITL" {
        usecase "Onay Bekleyen Eşleşmeleri İncele" as UC70
    }
}
```



```
}  
}
```

```
TK --> UC1  
TK --> UC3  
TK --> UC4  
TK --> UC5  
TK --> UC10  
TK --> UC11  
TK --> UC20  
TK --> UC23  
TK --> UC24  
TK --> UC25  
TK --> UC30  
TK --> UC31  
TK --> UC32  
TK --> UC40  
TK --> UC41  
TK --> UC43
```

```
IOT --> UC42  
OSB --> UC50  
OSB --> UC51  
OSB --> UC52  
OSB --> UC3
```

```
ADM --> UC2  
ADM --> UC3  
ADM --> UC60  
ADM --> UC61  
ADM --> UC62  
ADM --> UC63
```

```
UZM --> UC70  
UZM --> UC3
```

```
UC10 ..> UC12 : <<include>>  
UC12 ..> UC13 : <<include>>  
UC20 ..> UC21 : <<include>>  
UC20 ..> UC22 : <<include>>  
UC22 ..> UC70 : <<extend>>\n(confidence<0.80)  
UC40 ..> CLD : <<uses>>  
UC31 ..> UC30 : <<extend>>
```

```
@endum1
```

6.2 ER Diagram (erDiagram)

Bölüm 2.3-2.5'teki karar (user_role fix, material_class ENUM, UUID) ve bölüm 3.1'deki yeni tablolar (notifications, weights_config, audit_log, carbon_factors, human_review_queue, api_keys) dahil edilmiş haldedir.

erDiagram

```
osbs ||--o{ facilities : "kapsar"
facilities ||--o{ users : "kullanıcıları"
facilities ||--o{ inputs : "ihtiyaçları"
facilities ||--o{ outputs : "ürünleri"
facilities ||--o{ sensor_data : "sensörleri"
facilities ||--o{ facility_verification : "doğrulaması"
users ||--o{ messages : "mesajları"
users ||--o{ notifications : "bildirimleri"
users ||--o{ api_keys : "anahtarları"
users ||--o{ audit_log : "aksiyonları"
outputs ||--|| material_passports : "pasaportu"
outputs ||--o{ matches : "arz"
inputs ||--o{ matches : "talep"
outputs ||--o{ embeddings : "vektörü_output"
inputs ||--o{ embeddings : "vektörü_input"
matches ||--o{ reports : "raporları"
matches ||--o{ human_review_queue : "HITL"
```

```
osbs {
    uuid id PK
    varchar name
    varchar city
    geography region "POLYGON (PostGIS)"
    timestamp created_at
}
```

```
facilities {
    uuid id PK
    varchar name
    varchar tax_id UK
    varchar sector
    geography location "POINT (PostGIS)"
    uuid osb_id FK
    boolean verified
    timestamp created_at
    timestamp updated_at
}
```

```
users {
    uuid id PK
    uuid facility_id FK
    varchar email UK
    varchar password_hash
    user_role role "ENUM(user,admin,osb_manager)"
    varchar refresh_token
    timestamp last_login
    timestamp created_at
}
```

```
inputs {
    uuid id PK
    uuid facility_id FK
    material_class material_class "ENUM"
    text description
    jsonb specs
    decimal quantity_kg
    varchar frequency
    timestamp created_at
}
```

```
outputs {
    uuid id PK
    uuid facility_id FK
```

```
material_class material_class "ENUM"
text description
jsonb composition
decimal quantity_kg
decimal stock
timestamp created_at
}

embeddings {
  uuid id PK
  uuid record_id "polymorphic FK"
  record_type record_type "ENUM(input,output)"
  vector vector "VECTOR(768) pgvector, HNSW"
  varchar model_version
  timestamp created_at
}

matches {
  uuid id PK
  uuid output_id FK
  uuid input_id FK
  integer total_score "0-100"
  jsonb breakdown "material,quality,env,logistics,economic 0-100"
  match_status status "ENUM(pending,accepted,rejected,completed)"
  decimal co2_saved
  decimal cost_saving
  decimal cbam_impact
  text rejection_reason
  timestamp expires_at
  timestamp created_at
}

material_passports {
  uuid id PK
  uuid output_id FK "UK"
  jsonb passport_data "product_id,composition,origin,traceability"
  boolean dpp_compliant
  varchar qr_code
  varchar pdf_url
  timestamp created_at
}

reports {
  uuid id PK
  uuid match_id FK
  report_type report_type "ENUM(environmental,cbam,dpp)"
  jsonb data
  varchar pdf_url
  timestamp created_at
}

sensor_data {
  uuid id PK
  uuid facility_id FK
  varchar sensor_type
  decimal value
  varchar unit
  timestamp timestamp
}

messages {
  uuid id PK
  uuid user_id FK
  chat_role role "ENUM(user,assistant)"
  text content
  timestamp created_at
}

notifications {
  uuid id PK
  uuid user_id FK
```

```
    varchar type "match_accepted,match_rejected,review_required"
    varchar title
    text body
    jsonb payload
    timestamp read_at
    timestamp created_at
}

weights_config {
    uuid id PK
    integer version
    numeric material "0-1"
    numeric quality "0-1"
    numeric environmental "0-1"
    numeric logistics "0-1"
    numeric economic "0-1"
    boolean active
    uuid created_by FK
    timestamp created_at
}

audit_log {
    uuid id PK
    uuid actor_id FK
    varchar action "create,update,delete,accept,reject"
    varchar entity
    uuid entity_id
    jsonb before
    jsonb after
    inet ip_address
    timestamp created_at
}

carbon_factors {
    uuid id PK
    material_class material_class
    varchar factor_type "virgin,secondary,transport"
    numeric co2_per_kg "kg CO2e / kg"
    varchar source "Ecoinvent v3.10 vb."
    date valid_from
    date valid_to
    timestamp created_at
}

human_review_queue {
    uuid id PK
    uuid match_id FK
    numeric confidence "0-1"
    varchar reason
    varchar status "pending,approved,rejected"
    uuid reviewed_by FK
    timestamp reviewed_at
    text notes
    timestamp created_at
}

api_keys {
    uuid id PK
    uuid user_id FK
    varchar key_hash UK
    varchar name
    text[] scopes
    timestamp last_used
    timestamp expires_at
    timestamp created_at
    timestamp revoked_at
}

facility_verification {
    uuid id PK
    uuid facility_id FK
```

```
varchar document_type "tax_certificate,operating_permit"  
varchar document_url  
varchar status "pending,approved,rejected"  
uuid reviewed_by FK  
timestamp reviewed_at  
timestamp created_at  
}
```

6.3 Sequence Diagram

Mevcut "happy path" akışı korunmuş; eşiği 0.60'a düşürülmüş, Human-in-the-Loop dalı eklenmiş, bildirim akışı ve audit_log yazımları eklenmiştir.

```
sequenceDiagram
    autonumber
    actor TK as Tesis Kullanıcısı
    participant FE as Mobil/Web<br/>Frontend
    participant BE as Node.js<br/>Backend (NestJS)
    participant AI as Python AI<br/>Servisi (FastAPI)
    participant DB as PostgreSQL 16<br/>+ pgvector + PostGIS
    participant SC as ScoringEngine<br/>(Node.js)
    participant CB as CBAM Calculator
    participant NF as Notification<br/>Service
    participant WS as WebSocket<br/>Server

    rect rgb(232, 245, 233)
    note over TK,DB: 1) Çıktı Kaydı ve Vektörleştirme
    TK->>FE: Yeni çıktı gir (form)
    FE->>BE: POST /v1/materials/outputs
    BE->>DB: INSERT INTO outputs RETURNING id
    DB-->>BE: outputId (UUID)
    BE->>BE: buildEmbeddingText(output)
    BE->>AI: POST /embed {text, record_id, record_type='output'}
    AI->>AI: enrich_text() + SBERT encode
    AI-->>BE: {vector[768], model, dim, normalized}
    BE->>DB: INSERT INTO embeddings (record_id, record_type, vector)
    BE->>BE: generateDPP(output)
    BE->>DB: INSERT INTO material_passports
    DB-->>BE: passportId
    BE->>DB: INSERT INTO audit_log (action='create', entity='output')
    BE-->>FE: 201 Created {outputId, passportId, qrCode}
    FE-->>TK: Çıktı kaydedildi, QR gösterildi
    end

    rect rgb(224, 242, 254)
    note over TK,DB: 2) Eşleştirme Talebi
    TK->>FE: "Eşleştirmeleri bul" butonu
    FE->>BE: GET /v1/matches/find/:outputId
    BE->>DB: SELECT vector FROM embeddings WHERE record_id=:outputId
    DB-->>BE: outputVector
    BE->>DB: SELECT i.*, 1-(e_out.vector<=>e_in.vector) AS sim<br/>FROM inputs i JOIN embeddings e_in ON ...<br/>
    DB-->>BE: Top-20 aday
    end

    rect rgb(255, 243, 224)
    note over BE,CB: 3) Skor Hesaplama (her aday için)
    loop her aday için (max 20)
        BE->>DB: SELECT ST_Distance(f1.location, f2.location) AS distance_km
        DB-->>BE: distance_km
        BE->>DB: SELECT * FROM weights_config WHERE active=true
        DB-->>BE: weights
        BE->>SC: calculateMatchScore(output, candidate, similarity, distance_km, weights)
        SC->>SC: materialScore (30%)
        SC->>SC: qualityScore (20%)
        SC->>SC: environmentalScore (20%)
        SC->>SC: logisticsScore (15%)
        SC->>SC: economicScore (15%)
        SC->>CB: calculateCBAMImpact(match)
        CB->>DB: SELECT * FROM carbon_factors WHERE valid_to IS NULL
        DB-->>CB: factors
        CB-->>SC: {netReduction_kg, cbamSaving_eur}
        SC-->>BE: {totalScore, breakdown, co2Saved, costSaving, cbamImpact}
    end
    BE->>BE: Sırala (Top-10 seç)
    end

    alt confidence >= 0.80 (normal akış)
        BE->>DB: INSERT INTO matches (status='pending')
```

```
DB-->>BE: matchIds[]
BE-->>FE: 200 OK {matches: [...]}
FE-->>TK: Eşleştirme listesi
else confidence < 0.80 (HITL)
BE-->>DB: INSERT INTO human_review_queue (status='pending')
BE-->>NF: notify(uzman, 'review_required')
NF-->>DB: INSERT INTO notifications
NF-->>WS: push event
BE-->>FE: 200 OK {matches: [], pending_review: true}
FE-->>TK: "Eşleştirmeleriniz uzman onayında"
end

rect rgb(243, 229, 245)
note over TK,NF: 4) Eşleştirmeyi Kabul Etme
TK-->>FE: "Kabul Et" butonu
FE-->>BE: POST /v1/matches/:id/accept
BE-->>DB: UPDATE matches SET status='accepted'
BE-->>DB: INSERT INTO audit_log
BE-->>NF: notify(karşı taraf, 'match_accepted')
NF-->>DB: INSERT INTO notifications
NF-->>WS: push to counterparty
BE-->>FE: 200 OK {status:'accepted', contactInfo}
FE-->>TK: İletişim bilgileri açıldı
end

rect rgb(255, 235, 238)
note over TK,BE: 5) Eşleştirmeyi Reddetme (alternatif akış)
TK-->>FE: "Reddet" (gerekçe ile)
FE-->>BE: POST /v1/matches/:id/reject {reason}
BE-->>DB: UPDATE matches SET status='rejected', rejection_reason=$1
BE-->>DB: INSERT INTO audit_log
BE-->>NF: notify(karşı taraf, 'match_rejected')
BE-->>FE: 200 OK
end
```

6.4 Class Diagram

Bölüm 3.4'te tanımlanan yeni sınıflar (OSB, NotificationService, ChatbotService, IoTHandler, ReportEngine, ReviewQueueService, RuleEngine, AHPCalibrationService) dahil edilmiş; AI servisinin stateless tasarımı doğru gösterilmiştir (Embedding'i backend üretir).

```
classDiagram
%% ===== Domain Entities =====
class OSB {
    +UUID id
    +String name
    +String city
    +Polygon region
    +DateTime createdAt
    +getFacilities() List~Facility~
}

class Facility {
    +UUID id
    +String name
    +String taxId
    +String sector
    +Point location
    +UUID osbId
    +Boolean verified
    +DateTime createdAt
    +verify() void
    +getDistance(other: Facility) Number
}

class User {
    +UUID id
    +UUID facilityId
    +String email
    +String passwordHash
    +UserRole role
    +DateTime lastLogin
    +login(password: String) JWT
    +hasPermission(action: String) Boolean
}

class Input {
    +UUID id
    +UUID facilityId
    +MaterialClass materialClass
    +String description
    +JSONB specs
    +Number quantityKg
    +String frequency
    +buildEmbeddingText() String
}

class Output {
    +UUID id
    +UUID facilityId
    +MaterialClass materialClass
    +String description
    +JSONB composition
    +Number quantityKg
    +Number stock
    +buildEmbeddingText() String
    +isAvailable() Boolean
}

class Match {
    +UUID id
    +UUID outputId
    +UUID inputId
    +Number totalScore
    +JSONB breakdown
}
```



```
+MatchStatus status
+Number co2Saved
+Number costSaving
+Number cbamImpact
+DateTime expiresAt
+accept() void
+reject(reason: String) void
}

class Embedding {
+UUID id
+UUID recordId
+RecordType recordType
+Float[768] vector
+String modelVersion
+cosineSimilarity(other: Embedding) Number
}

class MaterialPassport {
+UUID id
+UUID outputId
+JSONB passportData
+Boolean dppCompliant
+String qrCode
+generateQR() String
+toPDF() Buffer
+isESPRCompliant() Boolean
}

class Report {
+UUID id
+UUID matchId
+ReportType reportType
+JSONB data
+String pdfUrl
+generatePDF() Buffer
+toEUFormat() JSON
}

class SensorData {
+UUID id
+UUID facilityId
+String sensorType
+Number value
+String unit
+DateTime timestamp
}

class ChatMessage {
+UUID id
+UUID userId
+ChatRole role
+String content
+DateTime createdAt
}

class Notification {
+UUID id
+UUID userId
+String type
+String title
+String body
+JSONB payload
+DateTime readAt
}

class ReviewQueueItem {
+UUID id
+UUID matchId
+Number confidence
+String status
}
```

```

        +UUID reviewedBy
        +DateTime reviewedAt
        +approve() void
        +reject(notes: String) void
    }

    class AuditLogEntry {
        +UUID id
        +UUID actorId
        +String action
        +String entity
        +UUID entityId
        +JSONB before
        +JSONB after
    }

    class WeightsConfig {
        +UUID id
        +Integer version
        +Number material
        +Number quality
        +Number environmental
        +Number logistics
        +Number economic
        +Boolean active
    }

    class CarbonFactor {
        +UUID id
        +MaterialClass materialClass
        +String factorType
        +Number co2PerKg
        +Date validFrom
        +Date validTo
    }

    %% ===== Services =====
    class ScoringEngine {
        <<service>>
        -WeightsConfig activeWeights
        +calculateMatchScore(output, candidate, similarity, distance) MatchResult
        +calculateMaterialScore() Number
        +calculateQualityScore() Number
        +calculateEnvironmentalScore() Number
        +calculateLogisticsScore() Number
        +calculateEconomicScore() Number
    }

    class CBAMCalculator {
        <<service>>
        +calculateCBAMImpact(match: Match) CBAMResult
        +getVirginFactor(mc: MaterialClass) Number
        +getSecondaryFactor(mc: MaterialClass) Number
    }

    class AIClient {
        <<service>>
        +String aiUrl
        +getEmbedding(text: String) Float[768]
        +classifyMaterial(text: String) Classification
        +healthCheck() HealthStatus
    }

    class EmbeddingsService {
        <<service>>
        -AIClient aiClient
        +createEmbedding(recordId, recordType, text) Embedding
        +findSimilar(vector, threshold, limit) List~Embedding~
    }

    class DPPGenerator {

```

```

    <<service>>
    +generate(output: Output) MaterialPassport
    +toPDF(passport) Buffer
    +validateESPR(passport) Boolean
}

class ReportEngine {
    <<service>>
    -CBAMCalculator cbam
    +generateEnvironmentalReport(match) Report
    +generateCBAMReport(match) Report
    +generateDPPReport(passport) Report
}

class NotificationService {
    <<service>>
    +send(userId, type, title, body, payload) Notification
    +pushWebSocket(userId, event) void
    +markAsRead(notifId) void
}

class ChatbotService {
    <<service>>
    -String claudeApiKey
    +sendMessage(userId, content) ChatMessage
    +getContext(userId, limit) List~ChatMessage~
}

class ReviewQueueService {
    <<service>>
    +enqueue(match, confidence, reason) ReviewQueueItem
    +approve(itemId, reviewerId) void
    +reject(itemId, reviewerId, notes) void
    +getPending() List~ReviewQueueItem~
}

class IoTHandler {
    <<service>>
    +subscribeMQTT(topic) void
    +onMessage(topic, payload) SensorData
}

class RuleEngine {
    <<service>>
    +evaluate(rule: String, context: JSONB) Boolean
    +reloadRules() void
}

class AHPCalibrationService {
    <<service>>
    +runCalibration(matches: List~Match~) WeightsConfig
    +activateWeights(configId) void
}

class AuditInterceptor {
    <<service>>
    +intercept(action, entity, before, after) AuditLogEntry
}

%% ===== Enums =====
class UserRole {
    <<enumeration>>
    USER
    ADMIN
    OSB_MANAGER
}

class MaterialClass {
    <<enumeration>>
    METAL
    PLASTIC

```

```

    ORGANIC
    CHEMICAL
    TEXTILE
    GLASS
    PAPER
    OTHER
}

class MatchStatus {
    <<enumeration>>
    PENDING
    ACCEPTED
    REJECTED
    COMPLETED
}

class RecordType {
    <<enumeration>>
    INPUT
    OUTPUT
}

class ReportType {
    <<enumeration>>
    ENVIRONMENTAL
    CBAM
    DPP
}

class ChatRole {
    <<enumeration>>
    USER
    ASSISTANT
}

%% ===== İlişkiler =====
OSB "1" o-- "*" Facility : kapsar
Facility "1" *-- "*" User : has
Facility "1" *-- "*" Output : produces
Facility "1" *-- "*" Input : declares
Facility "1" *-- "*" SensorData : sends
Output "1" o-- "1" MaterialPassport : has
Output "1" o-- "*" Match : as_supply
Input "1" o-- "*" Match : as_demand
Output "1" o-- "1" Embedding : indexed_by
Input "1" o-- "1" Embedding : indexed_by
Match "1" o-- "*" Report : generates
Match "1" o-- "0..1" ReviewQueueItem : HITL
User "1" o-- "*" ChatMessage : sends
User "1" o-- "*" Notification : receives

%% Servis bağımlılıkları
EmbeddingsService ..> AIClient : uses
EmbeddingsService ..> Embedding : creates
ScoringEngine ..> WeightsConfig : reads_from
ScoringEngine ..> CBAMCalculator : uses
CBAMCalculator ..> CarbonFactor : reads_from
DPPGenerator ..> MaterialPassport : creates
ReportEngine ..> Report : creates
ReportEngine ..> CBAMCalculator : uses
NotificationService ..> Notification : creates
ChatbotService ..> ChatMessage : creates
ReviewQueueService ..> ReviewQueueItem : manages
IoTHandler ..> SensorData : creates
AHPCalibrationService ..> WeightsConfig : creates
AuditInterceptor ..> AuditLogEntry : creates

```

Sonuç

Bu doküman DöngüNet projesinin mevcut MVP'sinden pilot OSB'ye hazır sisteme geçiş için gereken tüm mimari kararları, veritabanı değişikliklerini, kod düzeyinde yapılacak işleri ve 4 UML diyagramının düzeltilmiş Mermaid kaynak kodlarını içerir.

Hemen Başlanacak İşler (Bu Hafta)

- Herkes: Bölüm 2'deki 7 tutarsızlığı ve alınan kararları ekip toplantısında onayla.
- Backend: Migration 002-005'i lokal ortamda çalıştır ve test et.
- AI (Hamid): settings.py'de MATCH_THRESHOLD=0.60 yap, EmbedRequest'i UUID kabul edecek şekilde güncelle.
- Frontend: api-contracts/ klasörünü inceleyip OpenAPI kontratlarına dayalı istemci tip tanımlarını üret.

Onaylanacak Kararlar (Karar Toplantısı Gündemi)

#	Karar	Kim onaylar
1	MATCH_THRESHOLD = 0.60	AI + Backend
2	Proje adı = DöngüNet, Takım adı = VectorMatch	Herkes + Danışman
3	UserRole = ENUM('user','admin','osb_manager')	Backend + AI
4	material_class = PostgreSQL ENUM (8 değer, lowercase)	Backend + AI
5	embeddings.record_id = UUID (AI servis stateless)	Backend + AI
6	Rol dağılımı revize (3 kod yazan + 3 diğer)	Herkes + Danışman
7	HITL: confidence < 0.80 için otomatik kuyruk	Backend + AI
8	5 yeni tablo (notifications, weights_config, audit_log, carbon_factors, HITL_q)	Backend

Doküman sonu. Mermaid kodlarını render etmek için: <https://mermaid.live> veya VS Code Mermaid Preview eklentisi. PlantUML için: <https://www.plantuml.com/plantuml>